SALES FORCASTING

🎯 Introduction

Welcome to this data-driven journey where we analyze four years of global retail transactions to uncover patterns, segment insights, and forecast sales. We leverage Python, Prophet, and SARIMA models to project future demand and assist in inventory, logistics, and strategy planning.

Whether you're a business analyst, data scientist, or retail strategist, this notebook demonstrates how to bridge data with impact.

📌 Problem Statement

The goal is to analyze historical sales data to extract business insights and forecast future demand. We focus on:

Identifying sales patterns over time and across customer segments

Recognizing seasonality and outliers in daily sales

Forecasting sales for the upcoming week using Prophet and SARIMA

Comparing model performance and recommending operational decisions

💼 About the Dataset

The dataset contains 9,800 retail transactions over four years from a global superstore. It includes order-level details across customer demographics, geography, product info, and order metrics. The dataset is clean, structured, and ideal for time-series and customer analytics.

🛠️ Project Workflow

Inspect and clean dataset (convert dates, drop irrelevant columns)

Visualize daily, yearly, and regional sales trends

Analyze product category and sub-category sales

Segment customers and evaluate segment-wise performance

Forecast future daily sales using Prophet and SARIMA

Compare models, interpret results, and guide strategy

SALES FORCASTING

Dataset Introduction

This dataset contains 9,800 retail transactions from a global superstore spanning across four years. It captures detailed information about each order, including:

Dates: Order and shipping dates

Customer Info: Customer ID, name, and segment

Geography: Country, city, state, postal code, and region

Product Info: Product ID, category, sub-category, and name

Order Details: Sales value, quantity, discount, and profit

The dataset is clean and well-structured, with only a few missing postal codes, making it ideal for robust business analytics and forecasting.

Load and Inspect the Dataset

Step 1: Import necessary libraries

import pandas as pd

Step 2: Load the dataset

```
from google.colab import files
uploaded= files.upload()
```

df = pd.read_csv(train.csv))

Step 3: Show basic information

print("Dataset shape:", df.shape)

print("\nFirst five rows:")

SALES FORCASTING

```
display(df.head())


# Step 4: Overview of column data types and missing values

print("\nDataset Info:")

df.info()


print("\nMissing Values:")

print(df.isnull().sum())
```

Dataset shape: (9800, 18)

First five rows:

Row ID Order ID Order Date Ship Date Ship Mode Customer ID Customer Name Segment CountryCity State Postal Code Region Product ID Category Sub-Category Product Name Sales

0 1 CA-2017-152156 08/11/2017 11/11/2017 Second Class CG-12520 Claire Gute Consumer United States Henderson Kentucky 42420.0 South FUR-BO-10001798 Furniture Bookcases Bush Somerset Collection Bookcase 261.9600

1 2 CA-2017-152156 08/11/2017 11/11/2017 Second Class CG-12520 Claire Gute Consumer United States Henderson Kentucky 42420.0 South FUR-CH-10000454 Furniture Chairs Hon Deluxe Fabric Upholstered Stacking Chairs,... 731.9400

2 3 CA-2017-138688 12/06/2017 16/06/2017 Second Class DV-13045 Darrin Van Huff Corporate United States Los Angeles California 90036.0 West OFF-LA-10000240 Office Supplies Labels Self-Adhesive Address Labels for Typewriters b... 14.6200

3 4 US-2016-108966 11/10/2016 18/10/2016 Standard Class SO-20335 Sean O'Donnell Consumer United States Fort Lauderdale Florida 33311.0 South FUR-TA-10000577 Furniture Tables Bretford CR4500 Series Slim Rectangular Table 957.5775

4 5 US-2016-108966 11/10/2016 18/10/2016 Standard Class SO-20335 Sean O'Donnell Consumer United States Fort Lauderdale Florida 33311.0 South OFF-ST-10000760 Office Supplies Storage Eldon Fold 'N Roll Cart System 22.3680

Dataset Info:

SALES FORCASTING

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9800 entries, 0 to 9799
Data columns (total 18 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Row ID        9800 non-null   int64
 1   Order ID      9800 non-null   object
 2   Order Date    9800 non-null   object
 3   Ship Date     9800 non-null   object
 4   Ship Mode     9800 non-null   object
 5   Customer ID   9800 non-null   object
 6   Customer Name 9800 non-null   object
 7   Segment       9800 non-null   object
 8   Country       9800 non-null   object
 9   City          9800 non-null   object
 10  State         9800 non-null   object
 11  Postal Code   9789 non-null   float64
 12  Region        9800 non-null   object
 13  Product ID    9800 non-null   object
 14  Category      9800 non-null   object
 15  Sub-Category  9800 non-null   object
 16  Product Name  9800 non-null   object
 17  Sales         9800 non-null   float64
dtypes: float64(2), int64(1), object(15)
memory usage: 1.3+ MB

Missing Values:
Row ID          0
Order ID        0
```

SALES FORCASTING

Order Date 0

Ship Date 0

Ship Mode 0

Customer ID 0

Customer Name 0

Segment 0

Country 0

City 0

State 0

Postal Code 11

Region 0

Product ID 0

Category 0

Sub-Category 0

Product Name 0

Sales 0

dtype: int64

Data Cleaning & Preprocessing

Goals:

Convert date columns to datetime format

Drop unnecessary columns (like Row ID)

Decide how to handle the missing postal codes

Ensure numerical data is correctly typed

(Optional) Create a new column for aggregated daily sales for forecasting

SALES FORCASTING

```
# Convert Order Date and Ship Date to datetime with dayfirst=True
df['Order Date'] = pd.to_datetime(df['Order Date'], dayfirst=True)
df['Ship Date'] = pd.to_datetime(df['Ship Date'], dayfirst=True)

# Drop 'Row ID' – not meaningful
df.drop(columns=['Row ID'], inplace=True)

# Check missing Postal Code entries
print("Missing postal codes:")
print(df[df['Postal Code'].isnull()][['City', 'State']].drop_duplicates())
Missing postal codes:
          City    State
2234  Burlington  Vermont
# Desired ZIP code to use for Burlington, Vermont
default_burlington_zip = 5401  # We search the internet for the Postal Code

# Impute missing postal codes specifically for Burlington, VT
mask = df['Postal Code'].isnull() & \
      (df['City'] == 'Burlington') & \
      (df['State'] == 'Vermont')
df.loc[mask, 'Postal Code'] = default_burlington_zip

# Recheck missing values across dataset
print("\nRemaining missing values after ZIP imputation:")
print(df.isnull().sum())
Remaining missing values after ZIP imputation:
Order ID        0
Order Date       0
```

SALES FORCASTING

Ship Date 0

Ship Mode 0

Customer ID 0

Customer Name 0

Segment 0

Country 0

City 0

State 0

Postal Code 0

Region 0

Product ID 0

Category 0

Sub-Category 0

Product Name 0

Sales 0

dtype: int64

EDA Step 1: Sales Trends Over Time

Objective:

Aggregate daily sales

Visualize overall trend

Optionally smooth using rolling average

Time-Series Analysis

```
import matplotlib.pyplot as plt
import seaborn as sns
```

SALES FORCASTING

```
# Set plot style
sns.set(style="whitegrid")
plt.figure(figsize=(14, 6))

# Group by order date and sum sales
daily_sales = df.groupby('Order Date')['Sales'].sum().reset_index()

# Sort by date to ensure correct plotting
daily_sales.sort_values('Order Date', inplace=True)

# Plot daily sales
plt.plot(daily_sales['Order Date'], daily_sales['Sales'], label='Daily Sales', alpha=0.6)

# Add rolling average (7-day)
daily_sales['Rolling Avg'] = daily_sales['Sales'].rolling(window=7).mean()
plt.plot(daily_sales['Order Date'], daily_sales['Rolling Avg'], color='red', label='7-Day Rolling Average')

# Titles and labels
plt.title("Daily Sales Over Time", fontsize=16)
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.tight_layout()
plt.show()
```

Analysis of Daily Sales Trend

📈 Overall Trend

The rolling average (red line) shows a mild upward trend over the 4-year span.

SALES FORCASTING

There’s increased sales activity in the second half of the dataset (2017–2018) compared to the earlier years (2015–2016).

🔁 Seasonality

There are recurring patterns of sales spikes:

Typically around end-of-year (Q4) → likely holiday season sales (e.g. November–December).

Some spikes mid-year (June–July) could indicate mid-year promotions.

🚀 Sales Spikes & Outliers

Several sharp, high-volume spikes (especially early 2015, late 2016, and 2018).

These spikes could be:

Bulk purchases

B2B transactions

Special campaigns or promotions

One extreme spike in early 2015 surpasses $25,000 in a single day — worth isolating later to investigate the product or customer behind it.

🛑 Volatility

The blue line (raw daily sales) shows high volatility.

Daily sales are often very low (near zero), with intermittent high-value days — suggesting lumpy demand, not steady daily flow.

This may affect how we approach forecasting — we'll likely need smoothing, aggregation, or specialized models like Prophet or XGBoost with calendar features.

SALES FORCASTING

EDA Step 2: Breakdown by Year

Objective:

Visualize total yearly sales

Compare year-over-year growth

```
# Extract year from Order Date
df['Year'] = df['Order Date'].dt.year

# Aggregate sales by year
yearly_sales = df.groupby('Year')['Sales'].sum().reset_index()

# Plot
plt.figure(figsize=(10, 5))
sns.barplot(data=yearly_sales, x='Year', y='Sales', palette='viridis')

# Labels and titles
plt.title("Total Sales by Year", fontsize=16)
plt.xlabel("Year")
plt.ylabel("Total Sales")
plt.tight_layout()
plt.show()
```

Yearly Sales Trend Analysis

Sales increased significantly over time, especially after 2016.

Growth pattern:

SALES FORCASTING

2015 → 2016: Slight drop (~2–4%)

2016 → 2017: Noticeable increase (≈ +30%)

2017 → 2018: Continued strong growth (≈ +20%)

This shows accelerating business performance, likely due to:

Expanded customer base

Better marketing or promotions

High-performing product categories or regions

EDA Step 3: Regional Sales Analysis

Objectives:

Compare total sales by region

Identify top-performing and underperforming regions

Visualize patterns clearly using bar plots

```
# Group sales by Region
region_sales = df.groupby('Region')['Sales'].sum().reset_index().sort_values(by='Sales', ascending=False)

# Plot
plt.figure(figsize=(10, 5))
sns.barplot(data=region_sales, x='Region', y='Sales', palette='magma')
```

SALES FORCASTING

```
# Labels and title
plt.title("Total Sales by Region", fontsize=16)
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.tight_layout()
plt.show()
```

Regional Sales Analysis

West region leads with the highest total sales, just above $700,000.

East follows closely — strong performance, nearly matching the West.

Central and South lag significantly, with the South generating the lowest sales (~$390,000).

Insight: West and East are the core revenue drivers, suggesting higher market maturity or stronger customer bases. Central and South could represent growth opportunities with targeted marketing or logistics improvements.

EDA Step 4: Category & Sub-Category Sales Analysis

Objectives:

Identify which categories drive the most revenue

Break down performance by sub-category

Spot niches with low sales (potential for improvement or removal)

```
# Total Sales by Category
```

SALES FORCASTING

```
category_sales = df.groupby('Category')['Sales'].sum().reset_index().sort_values(by='Sales', ascending=False)

# Plot
plt.figure(figsize=(8, 5))
sns.barplot(data=category_sales, x='Category', y='Sales', palette='Set2')
plt.title("Total Sales by Category", fontsize=16)
plt.xlabel("Category")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
```

Category-Level Sales Analysis

Technology is the top-selling category, generating the highest revenue (~$830,000).

Furniture and Office Supplies are close behind, but clearly lag Technology.

All three categories contribute significantly, suggesting a diverse product mix.

Insight: Technology is likely the most profitable or in-demand segment. However, the gap isn't huge — all categories are worth maintaining or expanding strategically.

EDA Step 5: Total Sales by Sub-Category

```
# Total Sales by Sub-Category
subcat_sales = df.groupby('Sub-Category')['Sales'].sum().reset_index().sort_values(by='Sales', ascending=False)

# Plot
plt.figure(figsize=(14, 6))
sns.barplot(data=subcat_sales, x='Sub-Category', y='Sales', palette='cubehelix')
```

SALES FORCASTING

```
plt.title("Total Sales by Sub-Category", fontsize=16)
plt.xlabel("Sub-Category")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Sub-Category Sales Analysis

Top Performers:

📱 Phones and 🪑 Chairs dominate, each generating over $300,000 in sales.

Followed by Storage, Tables, and Binders — strong secondary contributors.

Mid-Tier:

Items like Machines, Accessories, and Copiers also contribute meaningfully.

Low Performers:

🖼️ Art, ✉️ Envelopes, 🏷️ Labels, and 📎 Fasteners have very low sales, likely niche or low-demand products.

📌 Insight: Focus on expanding inventory and promotions around high-performing sub-categories. Consider evaluating pricing, marketing, or even phasing out low-sales items if margins are poor.

EDA Step 5: Customer Segment Analysis

Objectives:

SALES FORCASTING

Identify which customer segments drive the most revenue

Compare performance between segments

Guide targeting strategies (B2B vs B2C)

```
# Total Sales by Customer Segment

segment_sales = df.groupby('Segment')['Sales'].sum().reset_index().sort_values(by='Sales',
ascending=False)


# Plot

plt.figure(figsize=(8, 5))

sns.barplot(data=segment_sales, x='Segment', y='Sales', palette='pastel')


# Labels and title

plt.title("Total Sales by Customer Segment", fontsize=16)

plt.xlabel("Segment")

plt.ylabel("Sales")

plt.tight_layout()

plt.show()
```

Customer Segment Analysis

Consumers are the dominant revenue source, contributing over $1.1 million — nearly half of total sales.

Corporate customers follow with strong performance (~$670K).

Home Office accounts for the smallest share, around $430K.

SALES FORCASTING

📌 Insight: Consumer segment is clearly the core market, but Corporate still offers significant value. Home Office may benefit from targeted offers or better product-market fit strategies to increase engagement.

Forecasting Sales for the Next 7 Days Using Prophet Model

Objective:

Build a model to predict daily sales for the 7 days following the last date in the dataset.

Prepare Data for Forecasting

We'll start with a simple, interpretable baseline using:

Daily aggregated sales

A forecast method like Prophet (good for business data with seasonality)

```
# Import Prophet
from prophet import Prophet

# Step 1: Aggregate daily sales
ts_df = df.groupby('Order Date')['Sales'].sum().reset_index()

# Step 2: Rename columns to match Prophet's expected input
ts_df.columns = ['ds', 'y']  # ds = date, y = value to forecast

# Step 3: Initialize and fit the Prophet model
model = Prophet(daily_seasonality=True)
model.fit(ts_df)
```

SALES FORCASTING

```
# Step 4: Create future dataframe for next 7 days
future = model.make_future_dataframe(periods=7)
forecast = model.predict(future)

# Step 5: Plot forecast
model.plot(forecast)
plt.title("7-Day Sales Forecast")
plt.xlabel("Date")
plt.ylabel("Predicted Sales")
plt.tight_layout()
plt.show()
13:25:28 - cmdstanpy - INFO - Chain [1] start processing
13:25:28 - cmdstanpy - INFO - Chain [1] done processing
/tmp/ipykernel_35/1110610065.py:23: UserWarning: The figure layout has changed to tight
  plt.tight_layout()

# Get last date in original dataset
last_date = ts_df['ds'].max()

# Filter only the next 7 days
future_forecast = forecast[forecast['ds'] > last_date][['ds', 'yhat', 'yhat_lower', 'yhat_upper']].copy()

# Rename columns for clarity
future_forecast.rename(columns={
    'ds': 'Date',
    'yhat': 'Predicted Sales',
    'yhat_lower': 'Lower Bound',
    'yhat_upper': 'Upper Bound'
}, inplace=True)
```

SALES FORCASTING

```
# Calculate average predicted sales
avg_sales = future_forecast['Predicted Sales'].mean()


# Create a new row for the average
average_row = pd.DataFrame({
    'Date': ['Average'],
    'Predicted Sales': [round(avg_sales, 2)],
    'Lower Bound': [None],
    'Upper Bound': [None]
})


# Append the row to the forecast DataFrame
forecast_with_avg = pd.concat([future_forecast, average_row], ignore_index=True)


# Display
print("📅 7-Day Sales Forecast with Average:")
display(forecast_with_avg)
```

📅 7-Day Sales Forecast with Average:

/tmp/ipykernel_35/3495573381.py:27: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the relevant entries before the concat operation.

```
  forecast_with_avg = pd.concat([future_forecast, average_row], ignore_index=True)
```

/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1458: RuntimeWarning: invalid value encountered in greater

```
  has_large_values = (abs_vals > 1e6).any()
```

/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1459: RuntimeWarning: invalid value encountered in less

```
  has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()
```

SALES FORCASTING

/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1459: RuntimeWarning: invalid value encountered in greater

has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()

Date	Predicted Sales	Lower Bound	Upper Bound
0	2018-12-31 00:00:00	2309.544143	-370.365257	4830.447164
1	2019-01-01 00:00:00	2555.669237	-240.174114	5279.339419
2	2019-01-02 00:00:00	1982.695857	-887.772729	4673.274138
3	2019-01-03 00:00:00	1306.098382	-1341.105947	4018.889247
4	2019-01-04 00:00:00	1828.026648	-827.001814	4489.286916
5	2019-01-05 00:00:00	2176.085571	-479.638263	4991.468982
6	2019-01-06 00:00:00	1856.253093	-883.790064	4543.041172
7	Average	2002.050000	NaN	NaN

Prophet Model Forecast

The Prophet model predicts an average of ~2,002 units/day over the next 7 days. The forecast indicates stable sales without extreme volatility, guiding moderate restocking and staffing needs.

Forecasting Sales for the Next 7 Days Using Seasonal Autoregressive Integrated Moving Average (SARIMA)

!pip install statsmodels

!pip install pmdarima

Requirement already satisfied: statsmodels in /usr/local/lib/python3.11/dist-packages (0.14.4)

Requirement already satisfied: numpy<3,>=1.22.3 in /usr/local/lib/python3.11/dist-packages (from statsmodels) (1.26.4)

Requirement already satisfied: scipy!=1.9.2,>=1.8 in /usr/local/lib/python3.11/dist-packages (from statsmodels) (1.15.2)

Requirement already satisfied: pandas!=2.1.0,>=1.4 in /usr/local/lib/python3.11/dist-packages (from statsmodels) (2.2.3)

Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.11/dist-packages (from statsmodels) (1.0.1)

Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.11/dist-packages (from statsmodels) (25.0)

SALES FORCASTING

Requirement already satisfied: mkl_fft in /usr/local/lib/python3.11/dist-packages (from numpy<3,>=1.22.3->statsmodels) (1.3.8)

Requirement already satisfied: mkl_random in /usr/local/lib/python3.11/dist-packages (from numpy<3,>=1.22.3->statsmodels) (1.2.4)

Requirement already satisfied: mkl_umath in /usr/local/lib/python3.11/dist-packages (from numpy<3,>=1.22.3->statsmodels) (0.1.1)

Requirement already satisfied: mkl in /usr/local/lib/python3.11/dist-packages (from numpy<3,>=1.22.3->statsmodels) (2025.1.0)

Requirement already satisfied: tbb4py in /usr/local/lib/python3.11/dist-packages (from numpy<3,>=1.22.3->statsmodels) (2022.1.0)

Requirement already satisfied: mkl-service in /usr/local/lib/python3.11/dist-packages (from numpy<3,>=1.22.3->statsmodels) (2.4.1)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2025.2)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)

Requirement already satisfied: intel-openmp<2026,>=2024 in /usr/local/lib/python3.11/dist-packages (from mkl->numpy<3,>=1.22.3->statsmodels) (2024.2.0)

Requirement already satisfied: tbb==2022.* in /usr/local/lib/python3.11/dist-packages (from mkl->numpy<3,>=1.22.3->statsmodels) (2022.1.0)

Requirement already satisfied: tcmlib==1.* in /usr/local/lib/python3.11/dist-packages (from tbb==2022.*->mkl->numpy<3,>=1.22.3->statsmodels) (1.3.0)

Requirement already satisfied: intel-cmplr-lib-rt in /usr/local/lib/python3.11/dist-packages (from mkl_umath->numpy<3,>=1.22.3->statsmodels) (2024.2.0)

Requirement already satisfied: intel-cmplr-lib-ur==2024.2.0 in /usr/local/lib/python3.11/dist-packages (from intel-openmp<2026,>=2024->mkl->numpy<3,>=1.22.3->statsmodels) (2024.2.0)

Collecting pmdarima

Downloading pmdarima-2.0.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl.metadata (7.8 kB)

Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (1.5.0)

SALES FORCASTING

Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (3.0.12)

Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (1.26.4)

Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (2.2.3)

Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (1.2.2)

Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (1.15.2)

Requirement already satisfied: statsmodels>=0.13.2 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (0.14.4)

Requirement already satisfied: urllib3 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (2.4.0)

Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (75.2.0)

Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.11/dist-packages (from pmdarima) (25.0)

Requirement already satisfied: mkl_fft in /usr/local/lib/python3.11/dist-packages (from numpy>=1.21.2->pmdarima) (1.3.8)

Requirement already satisfied: mkl_random in /usr/local/lib/python3.11/dist-packages (from numpy>=1.21.2->pmdarima) (1.2.4)

Requirement already satisfied: mkl_umath in /usr/local/lib/python3.11/dist-packages (from numpy>=1.21.2->pmdarima) (0.1.1)

Requirement already satisfied: mkl in /usr/local/lib/python3.11/dist-packages (from numpy>=1.21.2->pmdarima) (2025.1.0)

Requirement already satisfied: tbb4py in /usr/local/lib/python3.11/dist-packages (from numpy>=1.21.2->pmdarima) (2022.1.0)

Requirement already satisfied: mkl-service in /usr/local/lib/python3.11/dist-packages (from numpy>=1.21.2->pmdarima) (2.4.1)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.19->pmdarima) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.19->pmdarima) (2025.2)

SALES FORCASTING

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.19->pmdarima) (2025.2)

Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.22->pmdarima) (3.6.0)

Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.11/dist-packages (from statsmodels>=0.13.2->pmdarima) (1.0.1)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas>=0.19->pmdarima) (1.17.0)

Requirement already satisfied: intel-openmp<2026,>=2024 in /usr/local/lib/python3.11/dist-packages (from mkl->numpy>=1.21.2->pmdarima) (2024.2.0)

Requirement already satisfied: tbb==2022.* in /usr/local/lib/python3.11/dist-packages (from mkl->numpy>=1.21.2->pmdarima) (2022.1.0)

Requirement already satisfied: tcmlib==1.* in /usr/local/lib/python3.11/dist-packages (from tbb==2022.*->mkl->numpy>=1.21.2->pmdarima) (1.3.0)

Requirement already satisfied: intel-cmplr-lib-rt in /usr/local/lib/python3.11/dist-packages (from mkl_umath->numpy>=1.21.2->pmdarima) (2024.2.0)

Requirement already satisfied: intel-cmplr-lib-ur==2024.2.0 in /usr/local/lib/python3.11/dist-packages (from intel-openmp<2026,>=2024->mkl->numpy>=1.21.2->pmdarima) (2024.2.0)

Downloading pmdarima-2.0.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl (2.2 MB)

━━

2.2/2.2 MB 21.7 MB/s eta 0:00:0000:010:01

Installing collected packages: pmdarima

Successfully installed pmdarima-2.0.4

```
import pandas as pd

import matplotlib.pyplot as plt

from statsmodels.tsa.statespace.sarimax import SARIMAX


# Prepare the daily sales time series

ts_df = df.groupby('Order Date')['Sales'].sum().reset_index()

ts_df.set_index('Order Date', inplace=True)


# Ensure index is datetime
```

SALES FORCASTING

```
ts_df.index = pd.to_datetime(ts_df.index)

# Optional: Plot to visualize
ts_df['Sales'].plot(figsize=(14, 5), title="Daily Sales Time Series")
plt.ylabel("Sales")
plt.show()
```

Automatically Select SARIMA Parameters

Why use auto_arima?

It performs a stepwise search over ARIMA/SARIMA order combinations

Selects optimal parameters based on AIC, BIC, etc.

Handles differencing terms and seasonality intelligently

```
import pmdarima as pm

# Prepare your series (ts_df from earlier, indexed by date, daily frequency)
y = ts_df['Sales']

# Run auto_arima with daily seasonality (m=7)
auto_model = pm.auto_arima(
    y,
    seasonal=True,
    m=7,            # weekly seasonality
    stepwise=True,
    trace=True,
    error_action='ignore',
```

SALES FORCASTING

```
    suppress_warnings=True
)


# View best model details
print("✅ Best SARIMA model:", auto_model.summary())
```

```
Performing stepwise search to minimize aic
 ARIMA(2,1,2)(1,0,1)[7] intercept   : AIC=22472.633, Time=5.41 sec
 ARIMA(0,1,0)(0,0,0)[7] intercept   : AIC=23227.473, Time=0.06 sec
 ARIMA(1,1,0)(1,0,0)[7] intercept   : AIC=22879.020, Time=0.36 sec
 ARIMA(0,1,1)(0,0,1)[7] intercept   : AIC=22466.690, Time=1.74 sec
 ARIMA(0,1,0)(0,0,0)[7]             : AIC=23225.473, Time=0.04 sec
 ARIMA(0,1,1)(0,0,0)[7] intercept   : AIC=22468.263, Time=0.55 sec
 ARIMA(0,1,1)(1,0,1)[7] intercept   : AIC=inf, Time=3.70 sec
 ARIMA(0,1,1)(0,0,2)[7] intercept   : AIC=22468.398, Time=5.45 sec
 ARIMA(0,1,1)(1,0,0)[7] intercept   : AIC=22466.577, Time=1.16 sec
 ARIMA(0,1,1)(2,0,0)[7] intercept   : AIC=22468.549, Time=6.68 sec
 ARIMA(0,1,1)(2,0,1)[7] intercept   : AIC=inf, Time=6.38 sec
 ARIMA(0,1,0)(1,0,0)[7] intercept   : AIC=23227.722, Time=0.20 sec
 ARIMA(1,1,1)(1,0,0)[7] intercept   : AIC=22466.526, Time=2.57 sec
 ARIMA(1,1,1)(0,0,0)[7] intercept   : AIC=22468.406, Time=1.30 sec
 ARIMA(1,1,1)(2,0,0)[7] intercept   : AIC=22490.203, Time=3.08 sec
 ARIMA(1,1,1)(1,0,1)[7] intercept   : AIC=22488.744, Time=4.21 sec
 ARIMA(1,1,1)(0,0,1)[7] intercept   : AIC=22488.490, Time=1.61 sec
 ARIMA(1,1,1)(2,0,1)[7] intercept   : AIC=22489.922, Time=6.15 sec
 ARIMA(2,1,1)(1,0,0)[7] intercept   : AIC=22469.716, Time=7.43 sec
 ARIMA(1,1,2)(1,0,0)[7] intercept   : AIC=22475.013, Time=3.38 sec
 ARIMA(0,1,2)(1,0,0)[7] intercept   : AIC=22475.723, Time=2.00 sec
 ARIMA(2,1,0)(1,0,0)[7] intercept   : AIC=22775.751, Time=0.85 sec
 ARIMA(2,1,2)(1,0,0)[7] intercept   : AIC=22471.558, Time=4.00 sec
```

SALES FORCASTING

```
ARIMA(1,1,1)(1,0,0)[7]          : AIC=22465.331, Time=1.80 sec
ARIMA(1,1,1)(0,0,0)[7]          : AIC=22467.102, Time=0.72 sec
ARIMA(1,1,1)(2,0,0)[7]          : AIC=22467.089, Time=2.93 sec
ARIMA(1,1,1)(1,0,1)[7]          : AIC=22488.471, Time=1.40 sec
ARIMA(1,1,1)(0,0,1)[7]          : AIC=22465.436, Time=1.51 sec
ARIMA(1,1,1)(2,0,1)[7]          : AIC=inf, Time=6.23 sec
ARIMA(0,1,1)(1,0,0)[7]          : AIC=22465.264, Time=0.92 sec
ARIMA(0,1,1)(0,0,0)[7]          : AIC=22466.806, Time=0.37 sec
ARIMA(0,1,1)(2,0,0)[7]          : AIC=22466.979, Time=2.71 sec
ARIMA(0,1,1)(1,0,1)[7]          : AIC=22466.714, Time=4.52 sec
ARIMA(0,1,1)(0,0,1)[7]          : AIC=22465.372, Time=0.99 sec
ARIMA(0,1,1)(2,0,1)[7]          : AIC=inf, Time=4.70 sec
ARIMA(0,1,0)(1,0,0)[7]          : AIC=23225.725, Time=0.15 sec
ARIMA(0,1,2)(1,0,0)[7]          : AIC=22465.485, Time=1.06 sec
ARIMA(1,1,0)(1,0,0)[7]          : AIC=22877.021, Time=0.25 sec
ARIMA(1,1,2)(1,0,0)[7]          : AIC=22468.978, Time=2.17 sec

Best model:  ARIMA(0,1,1)(1,0,0)[7]
Total fit time: 100.808 seconds
✅ Best SARIMA model:                                SARIMAX Results
==========================================================================================
=====
Dep. Variable:                          y   No. Observations:              1230
Model:           SARIMAX(0, 1, 1)x(1, 0, [], 7)   Log Likelihood           -11229.632
Date:                     Sun, 13 Jul 2025   AIC                      22465.264
Time:                             13:27:23   BIC                      22480.605
Sample:                                  0   HQIC                     22471.036
                                    - 1230
Covariance Type:                       opg
```

SALES FORCASTING

```
==============================================================================
              coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ma.L1        -0.9679      0.008   -121.565      0.000      -0.984      -0.952
ar.S.L7       0.0555      0.026      2.104      0.035       0.004       0.107
sigma2     5.049e+06   5.72e+04     88.321      0.000    4.94e+06    5.16e+06
===================================================================================
Ljung-Box (L1) (Q):                   1.69   Jarque-Bera (JB):          31836.59
Prob(Q):                              0.19   Prob(JB):                      0.00
Heteroskedasticity (H):               1.22   Skew:                          3.41
Prob(H) (two-sided):                  0.04   Kurtosis:                     26.99
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Fit SARIMA & Forecast Next 7 Days

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Fit SARIMA model
model = SARIMAX(
    ts_df['Sales'],
    order=(0, 1, 1),
    seasonal_order=(1, 0, 0, 7),
    enforce_stationarity=False,
    enforce_invertibility=False
)
sarima_result = model.fit(disp=False)

# Forecast next 7 days
```

SALES FORCASTING

```
forecast_steps = 7
sarima_forecast = sarima_result.get_forecast(steps=forecast_steps)

# Extract forecast and confidence intervals
forecast_mean = sarima_forecast.predicted_mean
conf_int = sarima_forecast.conf_int()

# Build forecast DataFrame
sarima_output = pd.DataFrame({
    'Date': pd.date_range(start=ts_df.index[-1] + pd.Timedelta(days=1), periods=forecast_steps),
    'Predicted Sales': forecast_mean.values,
    'Lower Bound': conf_int.iloc[:, 0].clip(lower=0).values,
    'Upper Bound': conf_int.iloc[:, 1].values
})

# Add average row
avg_row = pd.DataFrame({
    'Date': ['Average'],
    'Predicted Sales': [round(sarima_output['Predicted Sales'].mean(), 2)],
    'Lower Bound': [None],
    'Upper Bound': [None]
})

sarima_output = pd.concat([sarima_output, avg_row], ignore_index=True)

# Display the forecast
print("📊 7-Day Sales Forecast (SARIMA):")
display(sarima_output)
```

SALES FORCASTING

/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

```
  self._init_dates(dates, freq)
```

/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

```
  self._init_dates(dates, freq)
```

📊 7-Day Sales Forecast (SARIMA):

/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available. Prediction results will be given with an integer index beginning at `start`.

```
  return get_prediction_index(
```

/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available. In the next version, calling this method in a model without a supported index will result in an exception.

```
  return get_prediction_index(
```

/tmp/ipykernel_35/1498010907.py:37: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the relevant entries before the concat operation.

```
  sarima_output = pd.concat([sarima_output, avg_row], ignore_index=True)
```

/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1458: RuntimeWarning: invalid value encountered in greater

```
  has_large_values = (abs_vals > 1e6).any()
```

/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1459: RuntimeWarning: invalid value encountered in less

```
  has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()
```

/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1459: RuntimeWarning: invalid value encountered in greater

```
  has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()
```

Date	Predicted Sales	Lower Bound	Upper Bound
0	2018-12-31 00:00:00	3130.195511	0.0	7980.778077
1	2019-01-01 00:00:00	2936.523158	0.0	7789.647445

SALES FORCASTING

2	2019-01-02 00:00:00	2833.260540	0.0	7688.925218
3	2019-01-03 00:00:00	2798.354846	0.0	7656.558586
4	2019-01-04 00:00:00	2879.444135	0.0	7740.185611
5	2019-01-05 00:00:00	2948.393356	0.0	7811.671242
6	2019-01-06 00:00:00	2827.736418	0.0	7693.549394
7	Average	2907.700000	NaN	NaN

Interpretation

The SARIMA model predicts higher average sales than Prophet (~2,908 vs ~2,003).

It suggests a consistent daily demand between 2,800–3,100 units.

The confidence intervals are wider (upper bound up to ~7,900), which still reflects the historical volatility of the dataset.

SARIMA vs Prophet

```
# Prepare Prophet data (excluding average row)
prophet_clean = forecast_with_avg[forecast_with_avg['Date'] != 'Average'].copy()
prophet_clean['Date'] = pd.to_datetime(prophet_clean['Date'])

# Prepare SARIMA data (excluding average row)
sarima_clean = sarima_output[sarima_output['Date'] != 'Average'].copy()
sarima_clean['Date'] = pd.to_datetime(sarima_clean['Date'])

# Merge both forecasts on Date
comparison = pd.merge(
    sarima_clean[['Date', 'Predicted Sales']],
    prophet_clean[['Date', 'Predicted Sales']],
    on='Date',
    suffixes=('_SARIMA', '_Prophet')
```

SALES FORCASTING

)

Optional: Add difference column

comparison['Difference'] = comparison['Predicted Sales_SARIMA'] - comparison['Predicted Sales_Prophet']

Display

print("📊 SARIMA vs Prophet – 7-Day Forecast Comparison:")

display(comparison)

📊 SARIMA vs Prophet – 7-Day Forecast Comparison:

Date	Predicted Sales_SARIMA	Predicted Sales_Prophet	Difference
0	2018-12-31	3130.195511	2309.544143	820.651368
1	2019-01-01	2936.523158	2555.669237	380.853920
2	2019-01-02	2833.260540	1982.695857	850.564683
3	2019-01-03	2798.354846	1306.098382	1492.256464
4	2019-01-04	2879.444135	1828.026648	1051.417487
5	2019-01-05	2948.393356	2176.085571	772.307785
6	2019-01-06	2827.736418	1856.253093	971.483325

📊 Forecasting Conclusion: SARIMA vs Prophet

We compared two time-series models to forecast 7 days of retail sales:

Model	Avg. Predicted Sales	Forecast Nature	Strengths
SARIMA	~2,908 units/day	Higher, seasonality-aware	Captures recent trends & seasonality
Prophet	~2,003 units/day	Conservative, smoother	Easy to interpret, stable forecasts

🔍 Observations:

SARIMA consistently predicts higher sales, with differences of +380 to +1,492 units/day compared to Prophet.

Prophet is more cautious and may underpredict during high-variance or post-holiday periods.

Both models show strong seasonality, but SARIMA adapts better to recent high demand.

SALES FORCASTING

✅ Recommendation:

Use SARIMA when planning for growth or peak season logistics.

Use Prophet for baseline operations or risk-averse inventory decisions.

For balanced planning, consider averaging both models' predictions.

✅ Summary

Through a blend of exploratory data analysis and time-series modeling, we identified key revenue drivers across geography, categories, and segments. We also produced 7-day forecasts using Prophet and SARIMA, where SARIMA showed stronger sensitivity to seasonality and recent spikes.

This notebook demonstrates a complete retail analytics pipeline — from data cleaning to business-ready forecasting.